

AI Usage Log

Assessment: Ubulu Africa — QA Engineer Take-Home

Author: Solomon Eshiet

Date: 3 August 2026

Document Version: 1.0

1. Overview

This log records all AI-assisted work across the assessment: **Part 1** (test strategy and spec analysis), **Part 3** (OrangeHRM UI automation), **Part 5** (GitHub Actions CI/CD), and **Part 6** (AI-assisted QA critique and utility).

Tools used:

Tool	Parts used
Cursor IDE (Claude / Composer agent)	Parts 1, 3, 5, 6 — primary assistant for drafting, scaffolding, debugging, and documentation
OpenAI API (gpt-4o-mini)	Part 6B — programmatic test-data generation via <code>ai/test-data-generator.ts</code>

General policy: AI **assisted** all listed tasks. Every deliverable was **human-reviewed, corrected, and verified** before submission. AI output was treated as an untrusted draft: oracles, locators, workflow behaviour, and financial limit values were validated against the spec, live application behaviour, or CI run results. No AI-generated test case, workflow file, or test-data file was accepted without explicit review.

2. Usage Log

Part	Task	AI Tool Used	What I Used It For	What I Corrected or Improved
Part 1	Ambiguity extraction	Cursor (Claude)	Generated initial open-questions list from the wallet spec (47 items across 10 categories)	Corrected:
Part 1	Test case generation	Cursor (Claude)	Drafted TC-001–TC-020 skeleton and coverage matrix (M-01–M-15)	Corrected:

Part 3	Page Object scaffolding	Cursor (Claude)	Generated boilerplate Page Object classes, Playwright config, and spec file structure	Corrected: _____
Part 5	GitHub Actions YAML	Cursor (Claude)	Generated workflow structure (lint, browser matrix, artifacts, sharding)	Corrected: _____
Part 6A	Spec test generation	Cursor (Claude)	Used raw AI output (25 test cases) as input for formal critique exercise — not executed as-is	N/A — output intentionally preserved unedited for critique (see docs/AI-Critique-Digital-Wallet-Test-Cases.pdf)
Part 6B	Utility script	Cursor (Claude) + OpenAI API	Generated initial ai/test-data-generator.ts structure, prompts, and validation logic	Corrected: _____

3. Example Prompts (Full Text)

3.1 Step 1.1 — Ambiguity Extraction (Part 1)

STEP 1.1 – Extract Ambiguities & Open Questions

I am a QA Engineer analysing a product specification before writing any tests. Below is the full feature specification. Your job is to help me identify every ambiguity, contradiction, gap, and unhandled edge case in it.

SPEC:

""

A mobile wallet app lets users send money to other wallet users and to bank accounts. Every account belongs to one of three tiers, based on the level of identity verification (KYC) completed.

1. Tier 1 accounts may send up to 20,000 per transaction and 50,000 per day.
Tier 2: 200,000 per transaction, 500,000 per day.
Tier 3: no per-transaction limit, 5,000,000 per day.
2. The daily limit resets at midnight.
3. Pending transfers count toward the daily limit. If a transfer fails or is reversed, the amount is returned to the user's available daily limit.
4. Transfers to a user's own linked bank accounts are exempt from all limits.
5. Users can schedule transfers for a future date; scheduled transfers execute at 8:00 AM on the scheduled day.
6. When a user completes the next KYC level, the tier upgrade takes effect immediately.
7. If a transfer would exceed the remaining daily limit, it is rejected with a clear error message showing the remaining limit.

""

For each issue found:

- Group by category (e.g. time/timezone, pending transfers, concurrency, scheduling, tier changes, exemptions, errors)
- State the issue clearly
- Explain why it matters for testing
- Write a specific open question for the product team

Be thorough. Do not propose solutions – only identify what is unclear or missing.

3.2 Step 6A.1 — Test Case Generation (Part 6A)

STEP 6A.1 – Generate AI Test Cases from Spec (for Critique)

You are a QA engineer. Below is a product specification for a Digital Wallet Transfer Limits feature. Generate test cases for this spec.

SPEC:

""

A mobile wallet app lets users send money to other wallet users and to bank accounts. Every account belongs to one of three tiers, based on the level of identity verification (KYC) completed.

1. Tier 1 accounts may send up to 20,000 per transaction and 50,000 per day.
Tier 2: 200,000 per transaction, 500,000 per day.
Tier 3: no per-transaction limit, 5,000,000 per day.
2. The daily limit resets at midnight.
3. Pending transfers count toward the daily limit. If a transfer fails or is reversed, the amount is returned to the user's available daily limit.
4. Transfers to a user's own linked bank accounts are exempt from all limits.
5. Users can schedule transfers for a future date; scheduled transfers execute

at 8:00 AM on the scheduled day.

6. When a user completes the next KYC level, the tier upgrade takes effect immediately.

7. If a transfer would exceed the remaining daily limit, it is rejected with a clear error message showing the remaining limit.

Generate 25 test cases covering:

- Happy paths for all 3 tiers
- Boundary conditions (at limit, just under, just over)
- Pending transfer scenarios
- Midnight reset
- Scheduled transfers
- Tier upgrade scenarios
- Own linked account exemption
- Error messages

Format each test case with: ID, title, preconditions, steps, expected result.

4. What I Corrected in AI Output

4.1 Example: AI was wrong — I caught it

In Part 6A, the AI generated **TC-AI-011** (*Pending transfer counts toward daily limit*) with a **25,000** transfer on a **Tier 1** account. Tier 1's per-transaction maximum is **20,000**, so the transfer should be rejected before reaching Pending state. The expected result assumed 25,000 reserved against the daily limit — an impossible precondition. I caught this during critique by cross-checking amounts against the spec's tier table, not by running the test. This is exactly the failure mode I document in Part 6A.2: structurally plausible test cases with incorrect oracles that would create false confidence if executed blindly.

4.2 Example: AI missed an important edge case

The AI's 25-case suite contained **no concurrency or race-condition scenarios** — for example, two simultaneous transfers that individually pass the remaining-limit check but together exceed the daily cap. My Part 1 risk register ranked this **#1** (TC-018; financial and regulatory exposure). The AI covered static boundaries well but omitted **temporal and parallel interaction** defects, which are typically the highest-severity gaps in limit-enforcement systems. I retained concurrency testing in my own suite and flagged the omission explicitly in the AI Critique document.

4.3 Example: AI output was good — I restructured it

For **Part 1**, the AI produced a strong raw list of open questions and a TC skeleton, but the output was **unformatted for submission**: no document control, no scope section, no automation split, and no explicit "blocked until product answers" gatekeeping. I restructured the material into a formal test plan with numbered sections (Introduction → Open Questions → Risk Prioritisation → Coverage Strategy → Automation vs Manual → Environment Assumptions), applied pairwise reduction rationale to trim the combination matrix from a naive 24+ cells to **15 core matrix cases plus edge suites**, and rewrote narrative sections in professional QA language suitable for a product team audience. The ideas were largely AI-assisted; the **engineering judgment and document architecture** were mine.

5. Reflection

AI materially **accelerates** drafting — open-question extraction, test-case skeletons, Page Object boilerplate, CI workflow structure, and edge-case data matrices would have taken significantly longer to type from a blank page. It is most valuable when the human has already defined what “good” looks like and uses AI to fill volume, not to replace thinking.

The primary **risk** is false confidence: polished, lengthy output reads authoritative even when oracles are wrong (TC-AI-011) or high-risk paths are absent (concurrency). In payments and limit-enforcement features, that risk has direct financial and compliance implications.

What **never leaves human hands** in my practice: signing off test oracles against ambiguous specs, prioritising risk, deciding what is blocked pending product answers, validating locators and behaviour against the real application, and approving CI results before declaring a pipeline trustworthy. AI is a drafting partner; accountability for quality remains with the QA engineer.

Document Control

Version	Date	Author	Changes
1.0	3 Aug 2026	Solomon Eshiet	Initial submission — Part 6C AI Usage Log

Related documents: Test Plan (Part 1); UI tests & CI (Parts 3, 5); AI Critique (Part 6A); AI Utility doc (Part 6B)

End of AI Usage Log